ARTICLE | Article in Press

Enhanced multiclass intrusion detection on imbalanced IoT NetFlow traffic using a three-tier stacked time-distributed LSTM with SMOTENC-based class balancing

Mustapha Ismail Kwari · N. Rajkumar · C. Selvarathi · Rabiu Aliyu Abdulkadir · Mideksa Tola Jiru

Received: 4 May 2026 / Accepted: 7 October 2026
© The Author(s) 2026

Abstract

The network-based attack surface has flourished abruptly with the unveiling of Internet of Things (IoT) devices. As a result, there is a pressing demand for automated intrusion detection systems (IDS) that can reliably identify a variety of attacks with severe class imbalance. Deep recurrent models are suitable for NetFlow traffic, however, a single-layer long short-term memory (LSTM) network is often inadequate for separating minority attack classes, and the effect of oversampling and model size is not well quantified. This study presents a three-tier stacked Time-Distributed long short-term memory network (ST-LSTM) evaluated on NF-BoT-IoT (600,100 flows, four attack classes and one benign class) under a leakage-free protocol. Ten flow features serve as predictors, while the class label and host identifiers are excluded. The data are divided into training, validation and test partitions before any fitting, and mixed-type oversampling (SMOTENC) is applied to the training partition only, at a ratio selected on validation data. On the untouched test set of 120,020 flows, the model reaches 83.98% accuracy, 0.8500 weighted F1, 0.6609 macro F1 and 0.9678 weighted ROC-AUC, the highest values among classical, shallow recurrent and stacked-ensemble baselines, and five-fold cross-validation gives 83.94 ± 0.19% accuracy. Standard SMOTE, by contrast, creates invalid protocol and flag codes and suppresses DDoS detection. A label-conflict analysis shows that 99.9% of DDoS and DoS flows share identical feature vectors with other classes, which limits attainable five-class accuracy to 84.91%, the model reaches 98.9% of this bound, and nearly all of its remaining errors fall on these ambiguous flows.

Keywords Stacked time-distributed LSTM (ST-LSTM) · Intrusion detection system (IDS) · Internet of things (IoT) · Network traffic classification · NF-BoT-IoT · SMOTENC · Class imbalance · Cross-validation · Data leakage

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-75619-5

Published online: 21 October 2026

Introduction

The Internet of Things (IoT) has grown rapidly, with billions of connected devices now deployed across smart homes, industrial plants, healthcare, and critical national infrastructure. This expansion has widened the threat landscape, exposing IoT deployments to denial-of-service (DoS), distributed denial-of-service (DDoS), reconnaissance scanning, and data-theft attacks[1,2] Network-based intrusion detection systems (NIDS) monitor traffic and raise alerts before an intrusion escalates, but the growing sophistication of IoT-specific attacks demands data-driven detectors that generalise beyond fixed signatures rather than rule sets curated by hand[3].

Machine-learning (ML) and deep-learning (DL) detectors have been studied extensively for this purpose[4,5], and lightweight models are attractive for resource-constrained IoT deployments[6]. Long short-term memory (LSTM) networks[7] are a common choice for traffic data because their gates regulate which information is retained between processing steps[8]. A single LSTM layer, however, provides a limited hierarchy of representations for separating several attack types whose flow statistics are similar.

Class imbalance is a second obstacle. Dominant traffic types outnumber rare attacks by orders of magnitude[9,10]. In the training partition of the NF-BoT-IoT data used here, Reconnaissance accounts for 78.43% of the records and Theft for 0.32%, a ratio of about 247:1. Classifiers trained naively on such distributions tend to ignore minority attacks, and ensemble strategies that combine complementary learners have been shown to mitigate, but not resolve, this effect in IoT IDS[11,12].

The third obstacle concerns evaluation. Reported IDS accuracies are frequently inflated by leakage, for instance when oversampling is performed before the data are split, when validation data are drawn from an oversampled set, or when identifiers and label-derived columns remain among the predictors[13,14]. Oversampling itself can be misapplied. The Synthetic Minority Over-sampling Technique (SMOTE) interpolates between neighbouring samples[15], when categorical fields such as protocol identifiers or TCP flags are label-encoded as integers, interpolation produces fractional codes that correspond to no real protocol and distorts nearest-neighbour distances. The SMOTENC variant, which treats nominal features separately, was proposed for exactly this situation[15], yet it is rarely used in NetFlow-based IDS studies.

This study addresses these three issues together. A three-tier stacked Time-Distributed LSTM (ST-LSTM) is trained on ten NetFlow predictors of NF-BoT-IoT with a strictly separated training, validation and test protocol. SMOTENC is applied to the training partition only, and the oversampling ratio, early stopping and all model choices are decided on a validation partition that is never oversampled. Because oversampling shifts the class frequencies seen during training, the network outputs are rescaled at inference to the class frequencies of the original training data[16], with small class offsets tuned on validation data. The study further measures how much of the remaining error is imposed by the data themselves: flows of different classes that share identical feature vectors cannot be separated by any classifier, and their prevalence defines an attainable accuracy ceiling.

The principal contributions of this study are as follows.

- A leakage-free evaluation protocol for NF-BoT-IoT in which the ten predictors, the target and the excluded columns are specified explicitly, the validation and test partitions are never oversampled, and all transformations are fitted on training data only.
- A three-tier ST-LSTM (482,053 parameters) trained with SMOTENC at a validation-selected partial ratio, which attains the highest test accuracy, weighted F1, macro F1 and ROC-AUC among the compared models and a cross-validated accuracy of 0.8394 ± 0.0019.
- Five-fold stratified cross-validation with mean ± standard deviation and 95% confidence intervals, one-vs-rest ROC-AUC analysis, per-class error-rate metrics (FNR, FDR, FPR), and comparison against five baselines under a shared preprocessing pipeline.
- Direct evidence that standard SMOTE is unsuitable for label-encoded NetFlow fields: at a matched ratio it creates 42, 1,104 and 849 invalid values in PROTOCOL, L7_PROTO and TCP_FLAGS and fails to predict a single DDoS flow correctly.

- A label-conflict analysis showing that DDoS and DoS flows are indistinguishable in the NetFlow feature space of NF-BoT-IoT, together with the attainable accuracy ceiling (84.91% on the test partition) against which the model is assessed.

The remainder of the paper is organised as follows. Section "Related work" reviews related work. Section "Materials and methods" describes the dataset, the leakage-free pipeline, the architecture, the class-frequency recalibration and the evaluation protocol. Section "Results" presents the results, Section "Discussion" discusses them, and Section "Conclusion" concludes the paper and states its limitations.

Related work

Deep learning has reshaped network intrusion detection by learning hierarchical features directly from traffic records[3,4]. Convolutional networks capture local structure in feature vectors, and hybrid CNN-LSTM designs add sequential modelling. *Halbouni et al.*[17] reported 95–97% accuracy with a CNN-LSTM on multi-class benchmarks, and *Altunay and Albayrak*[18] showed that a CNN+LSTM hybrid outperforms its individual components on industrial IoT traffic. Bidirectional LSTMs model dependencies in both directions[19], and *Ben Said et al.*[20] combined them with convolution and hybrid feature selection for software-defined networks, at the cost of doubling the recurrent computation. Hybrid deep-learning frameworks have been proposed for IIoT cloud environments[21], an attention-augmented CNN-LSTM has been evaluated on NSL-KDD and Bot-IoT traffic[22], and metaheuristic hyperparameter optimisation has been used to tune LSTM detectors[23]. Most of these studies report a single aggregate accuracy and do not isolate the effect of balancing or of individual architectural components.

A broad body of work pairs feature selection with neural or ensemble classifiers for wireless-sensor and IoT networks. Examples include conditional-random-field deep networks[24], ant-lion-optimised features with a gated recurrent unit[25], ensemble dimensionality reduction with random forests[26], nature-inspired optimisation for sensor networks[27] and cascaded LSTMs with hybrid feature selection[28]. Explainable and architecture-centred detectors extend these ideas to software-defined networks[29] and to deep belief networks for IoT[30], and explainable-AI components have been attached to CNN-LSTM and recurrent detectors for IoT and medical IoT settings[31,32]. Frameworks such as EIDM[33], optimisation-based IoT pipelines[34,35] and deep IoT detectors[36] report high aggregate accuracy on IoT benchmarks but seldom analyse minority-class behaviour. *Di Mauro et al.*[37] caution that feature-selection results in network IDS depend strongly on the evaluation protocol.

Class imbalance is usually handled by resampling[5]. SMOTE[15] is the most common choice; combined with LSTM it improves minority recall[38], and it has been paired with convolutional networks[39], clustering-based multi-stage classifiers[40] and enhanced random forests[10]. *Eid et al.*[41] showed that preprocessing and balancing choices change the ranking of ML models for IIoT intrusion detection. Boundary-focused variants such as Borderline-SMOTE[42] and ADASYN[43] change where samples are generated but still interpolate every feature as a continuous value. None of these methods is designed for integer-coded nominal fields, which is the setting SMOTENC addresses[15]. Resampling before partitioning, or drawing validation data from a resampled set, is a known source of optimistic bias[13,14].

NF-BoT-IoT is the NetFlow version of the Bot-IoT corpus[44], released with a standard set of flow features by *Sarhan et al.*[45]. Its extreme imbalance makes it a demanding benchmark for minority-class detection. Table 1 summarises representative studies and positions the present work, which differs by specifying the predictor set explicitly, applying SMOTENC only after partitioning at a validation-tuned ratio, reporting cross-validation with paired tests and per-class error rates, and quantifying the error that the feature space itself makes unavoidable.

Table 1 Representative deep-learning IDS studies and their evaluation scope.

Source	Dataset	Model	Balancing	CV/statistics	Per-class metrics and AUC
Laghrissi et al.[8]	KDD-99	LSTM + PCA	None	No	Partial
Halbouni et al.[17]	Multi-class benchmarks	CNN-LSTM	None	No	Partial
Altunay and Albayrak[18]	IIoT	CNN + LSTM	None	No	Partial
Ben Said et al.[20]	SDN	CNN-BiLSTM	Feature selection	No	Partial
Sayegh et al.[38]	IoT/CICIDS	LSTM	SMOTE	No	Partial
Alashjaee[22]	NSL-KDD, Bot-IoT	Attention-CNN-LSTM	None	No	Yes
This work	NF-BoT-IoT	Three-tier ST-LSTM	SMOTENC after partitioning; validation-tuned ratio	5-fold CV, 95% CI, paired tests	Yes; error rates and attainable ceiling

Materials and methods

Dataset and predictor specification

NF-BoT-IoT[45] contains 600,100 NetFlow records in 14 columns. The Attack column assigns each flow to one of five classes: Benign (13,859 flows), DDoS (56,844), DoS (56,833), Reconnaissance (470,655) and Theft (1,909). The corpus therefore contains four attack classes and one benign class. The Attack column is the prediction target and is never part of the model input. The Label column is a binary benign/attack flag derived from Attack, retaining it would reveal the target, so it is excluded. The source and destination IPv4 addresses identify hosts of the testbed rather than attack behaviour, and a model trained on them would learn which machines were attackers in this particular capture, both are excluded. The remaining ten columns form the predictor set, listed in Table 2. Every model in this study receives exactly these ten predictors, so the input shape of the recurrent models is (1, 10).

Fig. 1 portrays the Pearson correlation matrix of the ten predictors computed on the training partition. The strongest associations are structural. PROTOCOL and TCP_FLAGS are negatively correlated ($r = -0.613$), because flags are defined only for TCP flows; PROTOCOL and FLOW_DURATION_MILLISECONDS are also negatively related ($r = -0.405$). Packet and byte counts in the same direction are moderately correlated ($r = 0.540$ for incoming and 0.502 for outgoing traffic), and incoming and outgoing packet counts reach $r = 0.609$. All other pairs have $|r| \leq 0.236$. No pair is close to collinear, so all ten predictors were retained, in line with the observation that aggressive feature selection in network IDS is sensitive to the evaluation protocol[37].

Leakage-free partitioning and preprocessing

The complete pipeline is depicted in Fig. 2. The corpus was first divided by a stratified 80:20 split (random seed 42) into a development set of 480,080 flows and a test set of 120,020 flows. The development set was divided again by a stratified 90:10 split into a training partition of 432,072 flows and a validation partition of 48,008 flows. Table 3 gives the class counts. The validation partition was used for early stopping, for choosing the oversampling ratio, for every model and variant decision and for tuning the recalibration offsets. The test partition was used once, to evaluate the final configurations. Neither partition was oversampled.

The target was encoded alphabetically (0 = Benign, 1 = DDoS, 2 = DoS, 3 = Reconnaissance, 4 = Theft) with the scikit-learn label encoder[46]. The seven numerical predictors were standardised, $x' = (x - \mu)/\sigma$, with μ and σ estimated on the training partition and then applied unchanged to the validation and test partitions. The three

Table 2 Column roles in NF-BoT-IoT (categorical code counts refer to the training partition).

Column	Description	Type	Treatment
L4_SRC_PORT	Source transport-layer port	Numerical	Predictor; standardised
L4_DST_PORT	Destination transport-layer port	Numerical	Predictor; standardised
PROTOCOL	IP protocol identifier	Categorical (3 codes)	Predictor; nominal in SMOTENC
L7_PROTO	Application-layer protocol identifier	Categorical (87 codes)	Predictor; nominal in SMOTENC
IN_BYTES	Incoming bytes of the flow	Numerical	Predictor; standardised
OUT_BYTES	Outgoing bytes of the flow	Numerical	Predictor; standardised
IN_PKTS	Incoming packets of the flow	Numerical	Predictor; standardised
OUT_PKTS	Outgoing packets of the flow	Numerical	Predictor; standardised
TCP_FLAGS	Cumulative TCP flags	Categorical (23 codes)	Predictor; nominal in SMOTENC
FLOW_DURATION_MILLISECONDS	Flow duration in milliseconds	Numerical	Predictor; standardised
Attack	Traffic class	Target (5 classes)	Label-encoded output; never an input
Label	Binary benign/attack flag derived from Attack	Excluded	Removed to prevent target leakage
IPV4_SRC_ADDR, IPV4_DST_ADDR	Source and destination host addresses	Excluded	Removed as host identifiers

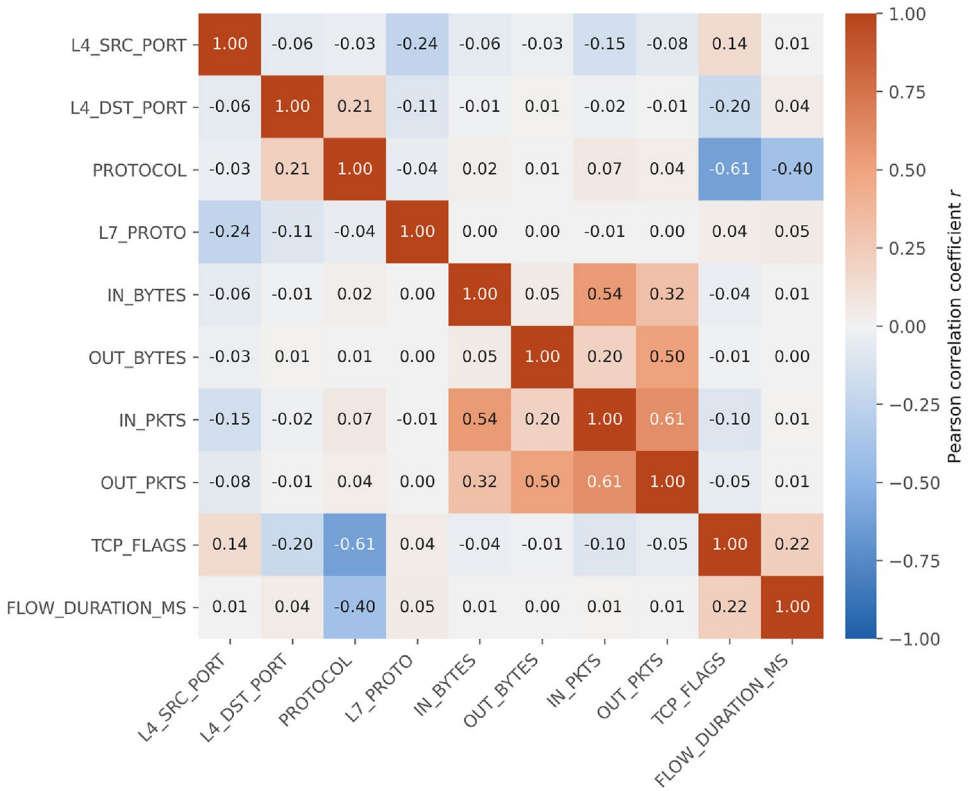


Fig. 1 Pearson correlation matrix of the ten predictors computed on the training partition.

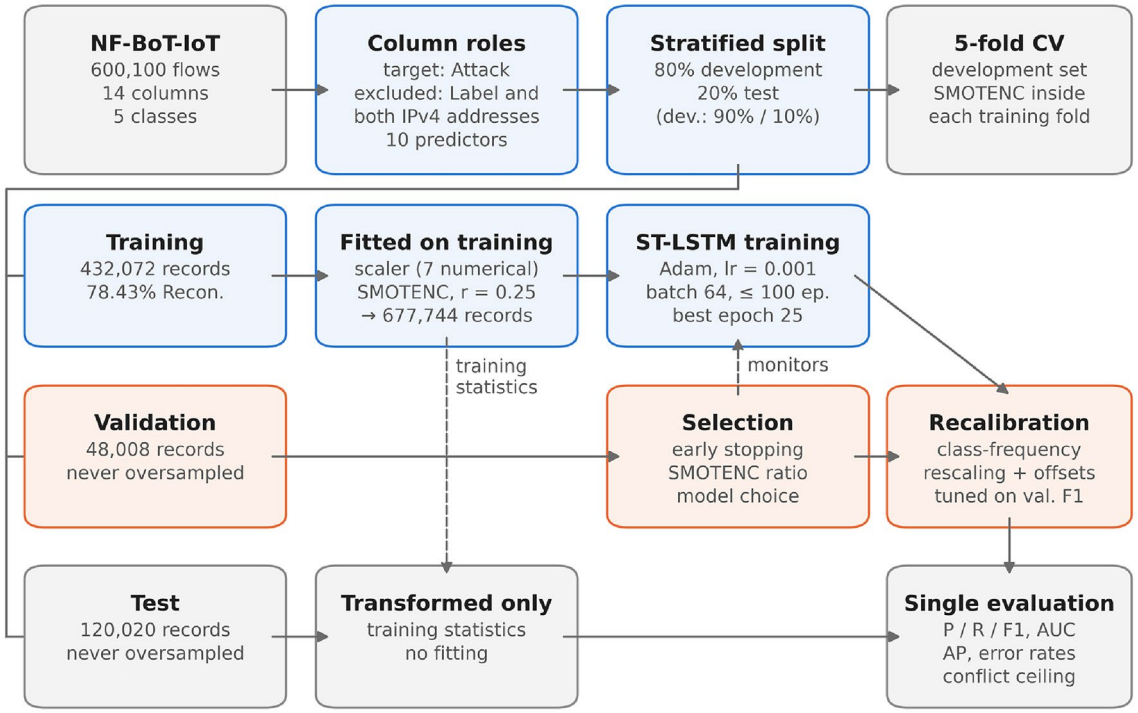


Fig. 2 Leakage-free workflow showing how the training, validation and test partitions are used.

Table 3 Class distribution of the leakage-free partitions of NF-BoT-IoT.

Class	Training (72%)	Validation (8%)	Test (20%)	Share of training (%)
Reconnaissance	338,872	37,652	94,131	78.43
DDoS	40,928	4,547	11,369	9.47
DoS	40,920	4,547	11,366	9.47
Benign	9,978	1,109	2,772	2.31
Theft	1,374	153	382	0.32
Total	432,072	48,008	120,020	100.00

categorical predictors were kept as integer codes and were not scaled, so that SMOTENC could treat them as nominal values. Each flow was reshaped into a sequence of one-time step with ten features.

Class balancing with SMOTENC

SMOTENC[15], as implemented in imbalanced-learn[47], was applied to the training partition after scaling. For a minority sample x_i and one of its $k = 5$ nearest minority-class neighbours x_j, the numerical part of a synthetic sample is

$$x_{new=xi+\lambda(xj-xi),\lambda\sim U(0,1)} \tag{1}$$

whereas each categorical feature of x_{new} takes the value that occurs most often among the k neighbours. Neighbours are found with a mixed distance in which every mismatching categorical feature adds a constant penalty,

$$d(x,y)^2 = \Sigma_{n\in N}(x_n - y_n)^2 + \Sigma_{c\in N} Med^2 \cdot 1[xc \neq yc] \tag{2}$$

where N and C are the sets of numerical and categorical predictors and Med is the median of the standard deviations of the numerical features within the minority class[15]. Synthetic samples therefore carry only category codes

that occur in the training data. A check after resampling confirmed that no new values appeared in PROTOCOL, L7_PROTO or TCP_FLAGS.

Full balancing would multiply the Theft class by a factor of about 247. A partial target was used instead: every class smaller than a fraction r of the majority class was raised to that size,

$$n'_c = \max\left(n_c, \lfloor r \cdot n_{\max} \rfloor\right), \tag{3}$$

where n_c is the training count of class c and $n_{\max} = 338{,}872$. The ratio was chosen on the validation partition from $r \in \{0.15, 0.25, 0.40\}$ (Section "Selection on the validation partition"), $r = 0.25$ was selected. Each minority class was raised to 84,718 records, giving 677,744 training records (Fig. 3). The multipliers were 8.49 for Benign, 2.07 for DDoS and DoS and 61.66 for Theft. For comparison, a standard-SMOTE set was generated at the same ratio; it was used only in the balancing ablation of Section "Standard SMOTE versus SMOTENC".

Three-tier stacked time-distributed LSTM

The architecture is displayed in Fig. 4 and listed layer by layer in Table 4. Each LSTM unit updates its state through input, forget and output gates[7]:

$$i_t = \sigma\left(W_i x_t + U_i h_{t-1} + b_i\right), f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right), o_t = \sigma\left(W_o x_t + U_o h_{t-1} + b_o\right), \tag{4}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot tanh\left(W_c x_t + U_c h_{t-1} + b_c\right), h_t = o_t \odot tanh\left(c_t\right), \tag{5}$$

where σ is the logistic function and ⊙ the element-wise product. The first tier (256 units, full sequence output) is followed by batch normalisation and dropout (0.3). A Time-Distributed dense projection (128 units, ReLU) then maps the 256-dimensional state to 128 dimensions, followed by batch normalisation and dropout (0.2). The second tier (128 units) is followed by a second projection to 64 dimensions, and the third tier (64 units) returns its final state, which passes through batch normalisation and dropout to a five-way softmax layer. The two projections reduce the representation between tiers, so that each recurrent tier operates on a compressed summary of the previous one. The model has 482,053 parameters, of which 480,773 are trainable, the 1,280 non-trainable parameters are the running statistics of the batch-normalisation layers.

Each flow is presented as a single time step. With $t = 1$, the terms $U\,h_{t-1}$ and $f_t \odot c_{t-1}$ in Eq. (4) and Eq. (5) vanish because the initial states are zero, and each tier acts as a gated non-linear transformation of its input rather than as a model of dependencies across flows. This choice matches per-flow scoring of streaming NetFlow

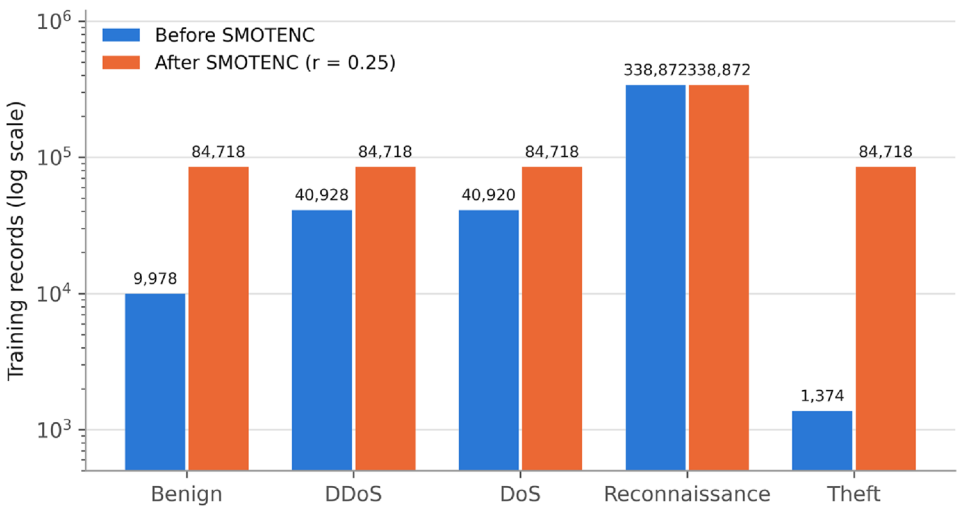


Fig. 3 Class counts of the training partition before and after SMOTENC oversampling with $r = 0.25$ (logarithmic axis).

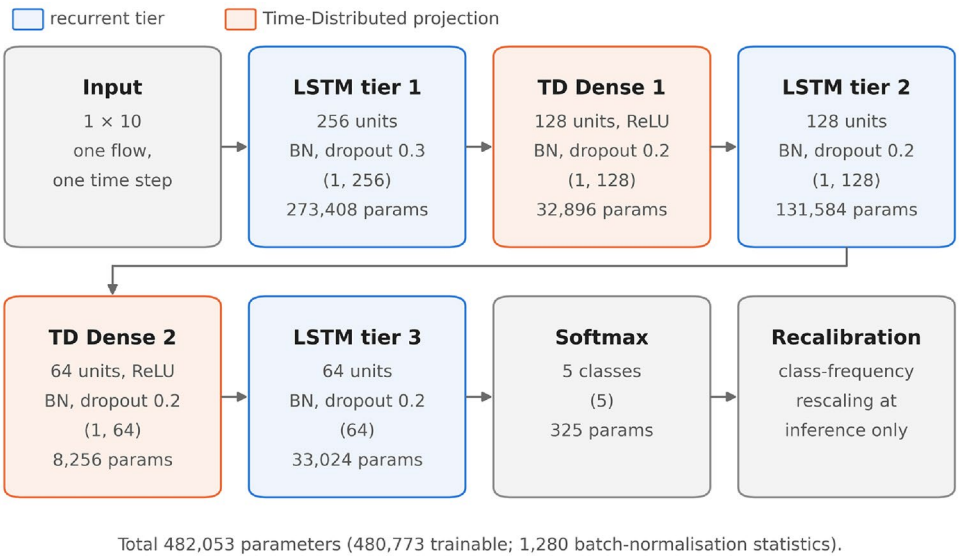


Fig. 4 Architecture of the three-tier ST-LSTM with the output shape and number of parameters of each block.

Table 4 Layer-by-layer summary of the three-tier ST-LSTM for the ten-predictor input.

Layer	Configuration	Output shape	Parameters
Input	shape = (1, 10)	(N, 1, 10)	—
LSTM tier 1	256 units, return_sequences = True	(N, 1, 256)	273,408
BatchNorm 1	momentum = 0.99	(N, 1, 256)	1,024
Dropout 1	rate = 0.3	(N, 1, 256)	—
TimeDistributed Dense 1	128 units, ReLU (256 → 128)	(N, 1, 128)	32,896
BatchNorm 2	momentum = 0.99	(N, 1, 128)	512
Dropout 2	rate = 0.2	(N, 1, 128)	—
LSTM tier 2	128 units, return_sequences = True	(N, 1, 128)	131,584
BatchNorm 3	momentum = 0.99	(N, 1, 128)	512
Dropout 3	rate = 0.2	(N, 1, 128)	—
TimeDistributed Dense 2	64 units, ReLU (128 → 64)	(N, 1, 64)	8,256
BatchNorm 4	momentum = 0.99	(N, 1, 64)	256
Dropout 4	rate = 0.2	(N, 1, 64)	—
LSTM tier 3	64 units, return_sequences = False	(N, 64)	33,024
BatchNorm 5	momentum = 0.99	(N, 64)	256
Dropout 5	rate = 0.2	(N, 64)	—
Output	Dense, softmax, 5 units	(N, 5)	325
Total			482,053 (480,773 trainable)

records. The Time-Distributed wrappers apply the same dense projection at every time step and allow the architecture to accept multi-flow windows without modification; windowed inputs are left to future work (Section "Conclusion").

Class-frequency recalibration at inference

Oversampling changes the class frequencies seen during training. With $r = 0.25$ they move from 0.0231, 0.0947, 0.0947, 0.7843 and 0.0032 (Benign, DDoS, DoS, Reconnaissance, Theft) to 0.1250 for each minority class and 0.5000 for Reconnaissance, so that the network over-predicts the smaller classes. At inference, the softmax

outputs were therefore rescaled to the class frequencies of the original training data, a standard adjustment for a change in class priors[16], and small class offsets b_c were added:

$$\hat{y}(x) = \arg\max_c [\log p(c|x) + \log \pi_c^{tr} - \log \pi_c^{os} + b_c], \tag{6}$$

where $p(c \mid x)$ is the softmax output and π_c^{tr} and π_c^{os} are the class frequencies of the original and the oversampled training partition. The offsets were chosen by coordinate search over $[-3, 3]$ in steps of 0.1 to maximise macro F1 on the validation partition, with $b_{\mathrm{Benign}} = 0$ as the reference; the selected values were −0.6 for Reconnaissance, +0.8 for Theft and 0 for DDoS and DoS. The recalibration acts on the softmax output only: it has no trainable parameters, leaves the network weights unchanged and costs one vector addition per flow. Probability-based metrics were computed from the renormalised recalibrated scores.

Training configuration and baselines

The models were implemented in Python 3.10 with TensorFlow/Keras, scikit-learn[46] and imbalanced-learn[47] and run on Google Colaboratory with an NVIDIA T4 GPU. Table 5 lists the settings. The network was trained with Adam[48] on sparse categorical cross-entropy, with early stopping on validation accuracy and learning-rate reduction on validation loss.

Five baselines were trained on the same partitions and evaluated on the same test set. Logistic Regression (saga solver, $C = 1$, 2,000 iterations) and LinearSVC ($C = 1$, 3,000 iterations, with sigmoid calibration by three-fold cross-validation for probability estimates) were trained on the SMOTENC-balanced training set. Random Forest (200 trees, maximum depth 25, minimum of three samples per leaf) was trained on a stratified 200,000-record subsample of that set to fit in memory. A single-layer LSTM with 64 units was trained with and without SMOTENC, and the ST-LSTM was also trained without balancing, giving a two-by-two design of depth and balancing. A stacked ensemble, in which a Random Forest is trained on the 64-dimensional penultimate representation of the ST-LSTM, was evaluated as an alternative to the recalibration. Its meta-learner used the same Random Forest settings and was fitted on the representations of the validation partition, balanced with standard SMOTE because these representations are continuous; in cross-validation it was fitted on each fold's early-stopping set and evaluated on the fold's held-out portion.

Evaluation protocol and label-conflict analysis

For each class c, precision, recall and F1 were computed from true positives (TP), false positives (FP) and false negatives (FN):

Table 5 Training and evaluation settings.

Setting	Value
Optimiser	Adam, learning rate 0.001, gradient clipping norm 1.0
Loss	Sparse categorical cross-entropy
Batch size/maximum epochs	64/100
Early stopping	Validation accuracy, patience 10, best weights restored
Learning-rate schedule	ReduceLROnPlateau on validation loss, factor 0.5, patience 5, minimum 10^{-6}
Oversampling	SMOTENC, $k = 5$, $r = 0.25$, training partition only
Random seed	42 (partitioning, oversampling and cross-validation folds)
Cross-validation	5 stratified folds of the development set; SMOTENC re-fitted inside each training fold; up to 80 epochs, early-stopping patience 8, learning-rate patience 4
Hardware and software	NVIDIA T4 GPU (Google Colaboratory); Python 3.10, TensorFlow/Keras, scikit-learn, imbalanced-learn

$$P_c = \frac{TP_C}{TP_C + FP_C}, R_C = \frac{TP_C}{TP_C + FN_C}, F1_C = \frac{2P_C R_C}{P_C + R_C}. \quad (7)$$

Macro averages weight all five classes equally, and weighted averages weight them by test support. Because the test set is dominated by Reconnaissance, macro F1 is the main criterion for minority-class detection. Per-class error rates were also reported,

$$FNR_c = \frac{FN_C}{TP_C + FN_C}, FDR_C = \frac{FP_C}{TP_C + FP_C}, FPR_C = \frac{FP_C}{FP_C + TN_C}, \quad (8)$$

together with one-versus-rest ROC-AUC and average precision (AP). Five-fold stratified cross-validation was run on the development set. Inside each fold, the training portion was split 90:10 into an inner training set and an early-stopping set, the scaler and SMOTENC were fitted on the inner training set, and the fold's held-out portion was used only for evaluation. For the recalibrated model, the offsets of each fold were tuned on that fold's early-stopping set. Results are reported as mean ± standard deviation over folds with a 95% confidence interval, mean ± 1.96·SD/√5, and the model with and without recalibration was compared with a paired *t*-test and a Wilcoxon signed-rank test over folds.

A label conflict occurs when the same ten-feature vector appears under more than one class. No classifier that sees only these features can assign different labels to identical inputs, so the best achievable decision for a vector v is its most frequent class. The attainable accuracy on a set of N flows is therefore;

$$A^* = \left(\frac{1}{N}\right) \Sigma_v max_c n_{v,c}, \quad (9)$$

where $n_{v,c}$ is the number of flows of class c with feature vector v. The ceiling was computed from exact matches of the raw predictor values over the full corpus, and test flows were identified by their positions in the stratified split. The same computation was repeated after merging DDoS and DoS and after removing the port fields, and each test flow was labelled as conflicting or unambiguous for a decomposition of the test error.

Results

Selection on the validation partition

All selection decisions were made on validation macro F1 (Table 6). Among the candidate oversampling ratios, $r = 0.25$ gave the highest validation macro F1 (0.5897) and was adopted. Balanced class weights, focal loss[49] with $\gamma = 2$, reduced dropout, a lower learning rate and wider or narrower networks did not improve on this reference. Two post-hoc strategies were then compared on the trained network: a stacked Random Forest on the penultimate representation, and the class-frequency recalibration of Section "Class-frequency recalibration at inference". The recalibration produced the highest validation macro F1 (0.6598) and is the proposed configuration, the stacked ensemble is retained as a comparison model.

Training behaviour

Fig. 5 displays the training and validation curves of the ST-LSTM trained with SMOTENC. Early stopping ended training after 35 epochs and restored the weights of epoch 25, where validation accuracy peaked at 0.8330 with a validation loss of 0.3180. Training accuracy (0.8007 at epoch 25) stays below validation accuracy throughout, because the training data contain a larger share of the harder minority classes and dropout is active only during training; the curves give no sign of over-fitting. The dotted line in the figure marks epoch 25. The validation loss shows isolated spikes, most visibly at epoch 18, that recover within one epoch.

Table 6 Validation-based selection of the oversampling ratio, training variants and post-hoc strategies.

Direction tested	Validation macro F1	Basis	Outcome
SMOTENC ratio $r = 0.15$	0.5363	Validation partition	Rejected
SMOTENC ratio $r = 0.25$	0.5897	Validation partition	Selected
SMOTENC ratio $r = 0.40$	0.5397	Validation partition	Rejected
Balanced class weights	0.5117	Validation partition	Rejected
Focal loss ($\gamma = 2$)	0.5716	Validation partition	Rejected
Reduced dropout	0.5470	Validation partition	Rejected
Reduced learning rate	0.5040	3-fold CV	Rejected
Wider network	0.5414	3-fold CV	No improvement
Narrower network	0.5395	3-fold CV	No improvement
Soft-voting ensemble	0.6120	Validation partition	Rejected
Stacked ensemble (ST-LSTM + Random Forest)	0.6171	Meta-validation split	Retained for comparison
Class-frequency recalibration with tuned offsets	0.6598	Validation partition	Selected (proposed)

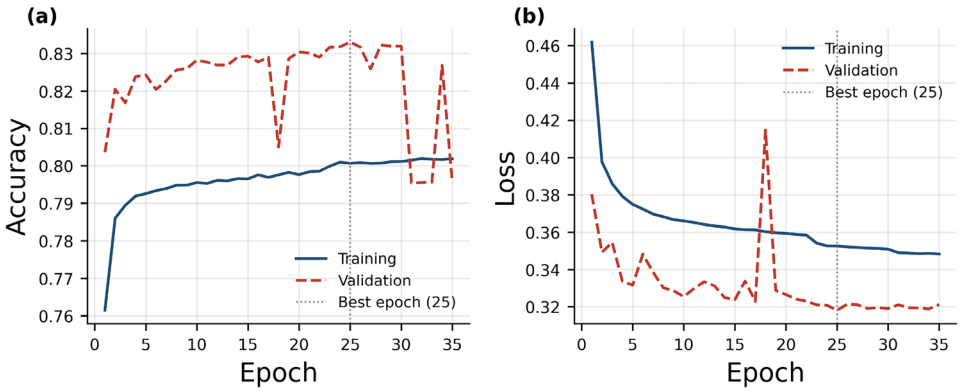


Fig. 5 Training and validation curves of the ST-LSTM trained with SMOTENC: (a) accuracy and (b) loss.

Table 7 Test-set performance of the ST-LSTM with SMOTENC before and after class-frequency recalibration ($n = 120{,}020$).

Configuration	Val. macro F1	Accuracy	Precision (w)	F1 (w)	F1 (macro)	ROC-AUC (w/macro)
Before recalibration	0.5896	0.8316	0.8709	0.8435	0.5876	0.9670/0.9646
Rescaled to original class frequencies	0.6401	0.8391	—	0.8491	0.6256	—
Rescaled with tuned offsets (proposed)	0.6598	0.8398	0.8753	0.8500	0.6609	0.9678/0.9648

Test-set performance of the proposed model

Table 7 reports the test performance of the ST-LSTM with SMOTENC before and after class-frequency recalibration. Without recalibration, the network reaches 83.16% accuracy and 0.5876 macro F1. Rescaling to the original class frequencies raises macro F1 to 0.6256, and the tuned offsets raise it to 0.6609, with accuracy 83.98%, weighted F1 0.8500 and weighted ROC-AUC 0.9678. The test gains in macro F1 (0.0733) is close to the validation gain (0.0702), which indicates that the offsets were not over-fitted to the validation partition. A dash in the table marks a metric that was not computed for the intermediate variant.

The per-class results are listed in Table 8, and the confusion matrix is demonstrated in Fig. 6, where each cell gives the count and the row percentage and the shading encodes the row-normalised proportion, which equals the class recall on the diagonal. Reconnaissance is detected with F1 0.9572 and a false discovery rate of 0.0116. Recalibration mainly benefits the two smallest classes. Theft F1 rises from 0.4663 to 0.7652 as its precision rises from 0.31 to 0.74, and Benign precision rises from 0.61 to 0.89. The price is a lower Benign recall (0.6353), since

Table 8 Per-class test performance of the proposed model.

Class	Precision	Recall	F1	Support	ROC-AUC	AP	FNR	FDR	FPR
Benign	0.8854	0.6353	0.7398	2,772	0.9909	0.8206	0.3647	0.1146	0.0019
DDoS	0.4471	0.2999	0.3590	11,369	0.9310	0.4577	0.7001	0.5529	0.0388
DoS	0.3688	0.7019	0.4836	11,366	0.9286	0.4447	0.2981	0.6312	0.1256
Reconnaissance	0.9884	0.9278	0.9572	94,131	0.9761	0.9939	0.0722	0.0116	0.0397
Theft	0.7390	0.7932	0.7652	382	0.9974	0.6656	0.2068	0.2610	0.0009
Macro average	0.6857	0.6716	0.6609	120,020	0.9648	0.6765	—	—	—
Weighted average	0.8753	0.8398	0.8500	120,020	0.9678	—	—	—	—

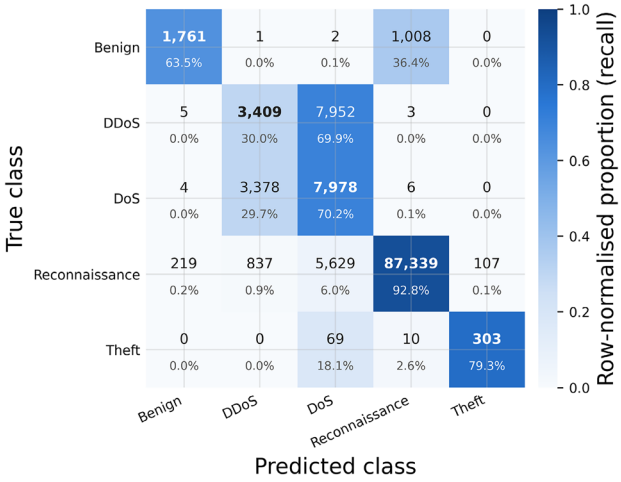


Fig. 6 Test-set confusion matrix of the proposed model with counts and row percentages.

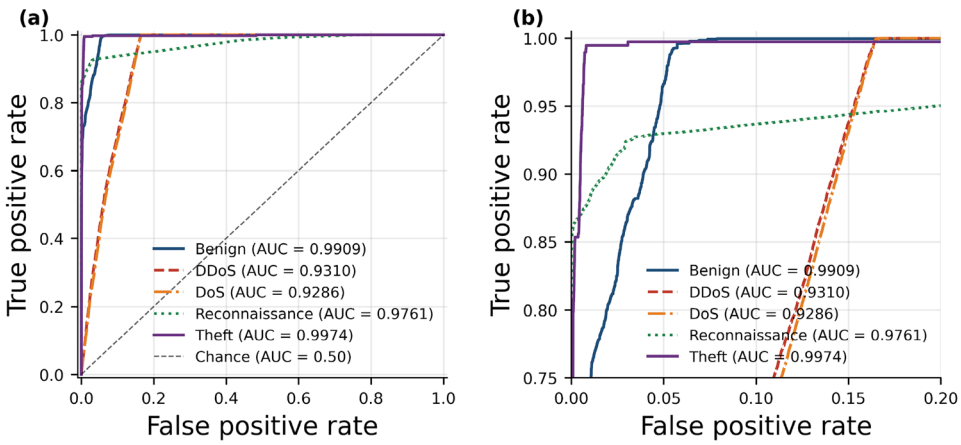


Fig. 7 One-versus-rest ROC curves of the proposed model on the test set: (a) full range and (b) enlarged region with false positive rate ≤ 0.20.

1,008 benign flows are assigned to Reconnaissance. DDoS and DoS remain the weakest classes (F1 0.3590 and 0.4836): 7,952 DDoS flows are predicted as DoS and 3,378 DoS flows as DDoS, and a further 5,629 Reconnaissance flows are assigned to DoS; 6,774 Reconnaissance test flows share their feature vector with flows of another class. Section "Label conflicts and the attainable ceiling" shows that this confusion is imposed by the data.

The one-versus-rest ROC curves (Fig. 7) confirm this pattern. Theft (AUC 0.9974) and Benign (0.9909) are separated almost perfectly, Reconnaissance reaches 0.9761, and DDoS (0.9310) and DoS (0.9286) have curves that almost coincide. The precision–recall curves (Fig. 8) are more informative for the rare classes. Reconnaissance has AP 0.9939, Benign 0.8206 and Theft 0.6656, whereas the DDoS and DoS curves stay close to a precision of 0.4 to 0.5 over most of the recall range, with AP 0.4577 and 0.4447. The horizontal lines in the precision–recall plot mark the prevalence of each class, which is the precision of a random classifier.

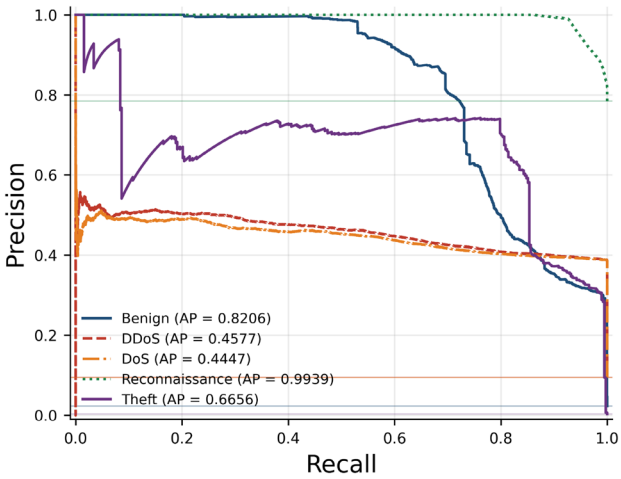


Fig. 8 Precision–recall curves of the proposed model on the test set.

Table 9 Five-fold stratified cross-validation on the development set (mean ± SD; 95% confidence interval in brackets).

Metric	ST-LSTM + SMOTENC, before recalibration	ST-LSTM + SMOTENC, recalibrated (proposed)	Stacked ensemble	Paired test, recalibrated vs before
Accuracy	0.8177 ± 0.0134	0.8394 ± 0.0019 [0.8378, 0.8411]	0.8222 ± 0.0049	$t = 3.61$, $p = 0.023$
F1 (weighted)	0.8139 ± 0.0173	0.8358 ± 0.0092 [0.8278, 0.8439]	0.8386 ± 0.0037	—
F1 (macro)	0.5103 ± 0.0513	0.6131 ± 0.0495 [0.5697, 0.6565]	0.6344 ± 0.0152	$t = 13.94$, $p < 0.001$
ROC-AUC (weighted)	0.9638 ± 0.0062	0.9672 ± 0.0016 [0.9658, 0.9686]	0.9638 ± 0.0005	—

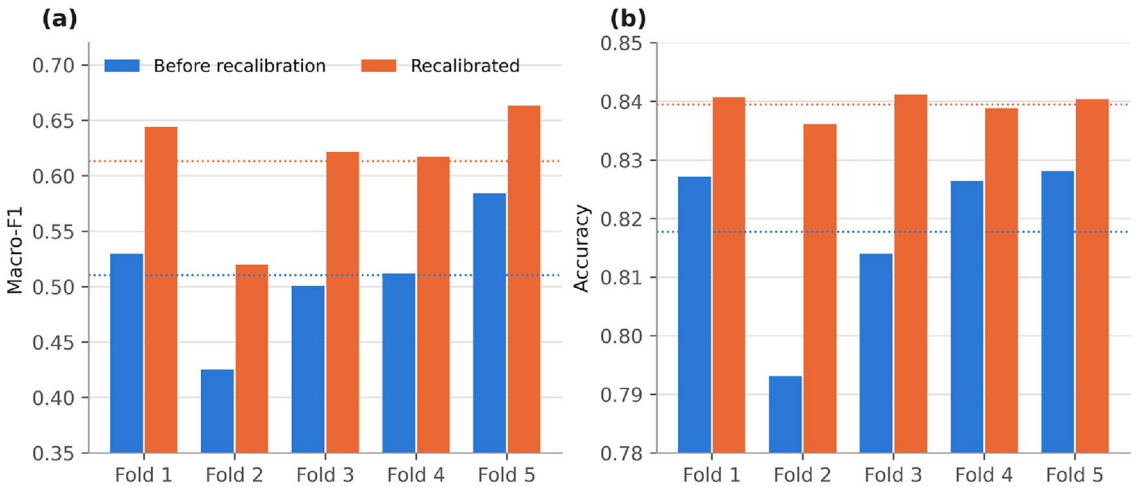


Fig. 9 Per-fold (**a**) macro F1 and (**b**) accuracy in five-fold cross-validation before and after class-frequency recalibration.

Cross-validation

Table 9 summarises five-fold cross-validation, and Fig. 9 reveals the results per fold. Class-frequency recalibration raised macro F1 in every fold, by between 0.0792 and 0.1208, from 0.5103 ± 0.0513 to 0.6131 ± 0.0495. The mean gain of 0.1028 is significant under a paired t-test ($t(4) = 13.94$, $p = 1.5 \times 10^{-4}$). Accuracy rose from 0.8177 ± 0.0134 to 0.8394 ± 0.0019 ($t(4) = 3.61$, $p = 0.023$), and its spread across folds fell by a factor of seven.

The Wilcoxon signed-rank test gives $p = 0.0625$ for both metrics, which is the smallest value attainable with five pairs. The stacked ensemble reaches a slightly higher mean macro F1 (0.6344 ± 0.0152) with lower variability, but lower accuracy (0.8222 ± 0.0049) and ROC-AUC (0.9638 ± 0.0005). Its mean macro F1 lies inside the 95% interval of the proposed model, so five folds do not resolve the difference in macro F1, whereas the proposed model has the higher accuracy and ROC-AUC with non-overlapping intervals. Dotted lines in Fig. 9 mark the means over folds.

Comparison with baselines

Table 10 compares all models on the same test set, and Fig. 10 illustrates their confusion matrices. The proposed model has the highest accuracy, weighted precision, weighted F1, macro F1 and weighted ROC-AUC. Among the classical models, Random Forest is the strongest (macro F1 0.6122) and detects Theft well (F1 0.6500), but it confuses DDoS and DoS in the same way as the recurrent models. Logistic Regression and LinearSVC reach macro F1 0.4751 and 0.3738; LinearSVC predicts only 693 DDoS flows correctly and assigns 6,029 DDoS and 6,021 DoS flows to Reconnaissance. In Fig. 10, panels (a)–(c) show Logistic Regression, LinearSVC and Random Forest trained with SMOTENC; (d) and (e) the single-layer LSTM without and with SMOTENC; (f) the ST-LSTM without balancing; (g) the ST-LSTM with SMOTENC before recalibration (labelled 'uncorrected'), and (h) the stacked ensemble. Counts are shown in each cell and the shading encodes the row-normalised proportion.

The two-by-two design of depth and balancing shows how the two factors interact. Without balancing, the three-tier network has a lower macro F1 than the single-layer LSTM (0.4313 against 0.4754): it assigns almost all DDoS and DoS flows to DDoS (DoS F1 0.0323) and never predicts Theft. SMOTENC raises macro F1 by 0.0906 for the single-layer LSTM and by 0.1563 for the three-tier network, so the deeper model benefits more from balancing, and only in combination with it does depth improve on the shallow model. Accuracy changes little in either case (−0.0180 and −0.0036), which shows that accuracy alone would hide these differences. These four configurations were evaluated on a single split and should be read as indicative.

Standard SMOTE versus SMOTENC

To test the concern that standard SMOTE is inappropriate for label-encoded categorical fields, the ST-LSTM was trained on a standard-SMOTE set generated at the same ratio ($r = 0.25$). Table 11 and Fig. 11 summarise the result. Standard SMOTE treated the category codes as continuous and created 42 values of PROTOCOL that do not exist in the training data (against 3 valid codes), 1,104 invalid values of L7_PROTO (against 87) and 849 of TCP_FLAGS (against 23). SMOTENC created none. The effect on detection is concentrated on the classes that depend most on these fields. With standard SMOTE, the network did not predict a single flow as DDoS, which set DDoS F1 to zero, and macro F1 fell from 0.5876 to 0.5277. Accuracy was almost unchanged (0.8300 against 0.8316), so an evaluation based on accuracy alone would not have revealed the problem.

Table 10 Test-set comparison of all models under identical partitions ($n = 120{,}020$).

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)	F1 (macro)	ROC-AUC (w)
Logistic Regression + SMOTENC	0.7950	0.8405	0.7950	0.8130	0.4751	0.9334
LinearSVC + SMOTENC	0.7605	0.7596	0.7605	0.7447	0.3738	0.9177
Random Forest + SMOTENC	0.8202	0.8570	0.8202	0.8356	0.6122	0.9661
Standard LSTM (no balancing)	0.8347	0.8623	0.8347	0.8382	0.4754	0.9618
Standard LSTM + SMOTENC	0.8167	0.8664	0.8167	0.8352	0.5660	0.9664
ST-LSTM (no balancing)	0.8352	0.8682	0.8352	0.8186	0.4313	0.9661
ST-LSTM + SMOTENC (before recalibration)	0.8316	0.8709	0.8316	0.8435	0.5876	0.9670
Stacked ensemble (ST-LSTM + Random Forest)	0.8169	0.8606	0.8169	0.8343	0.6149	0.9628
ST-LSTM + SMOTENC (proposed)	0.8398	0.8753	0.8398	0.8500	0.6609	0.9678

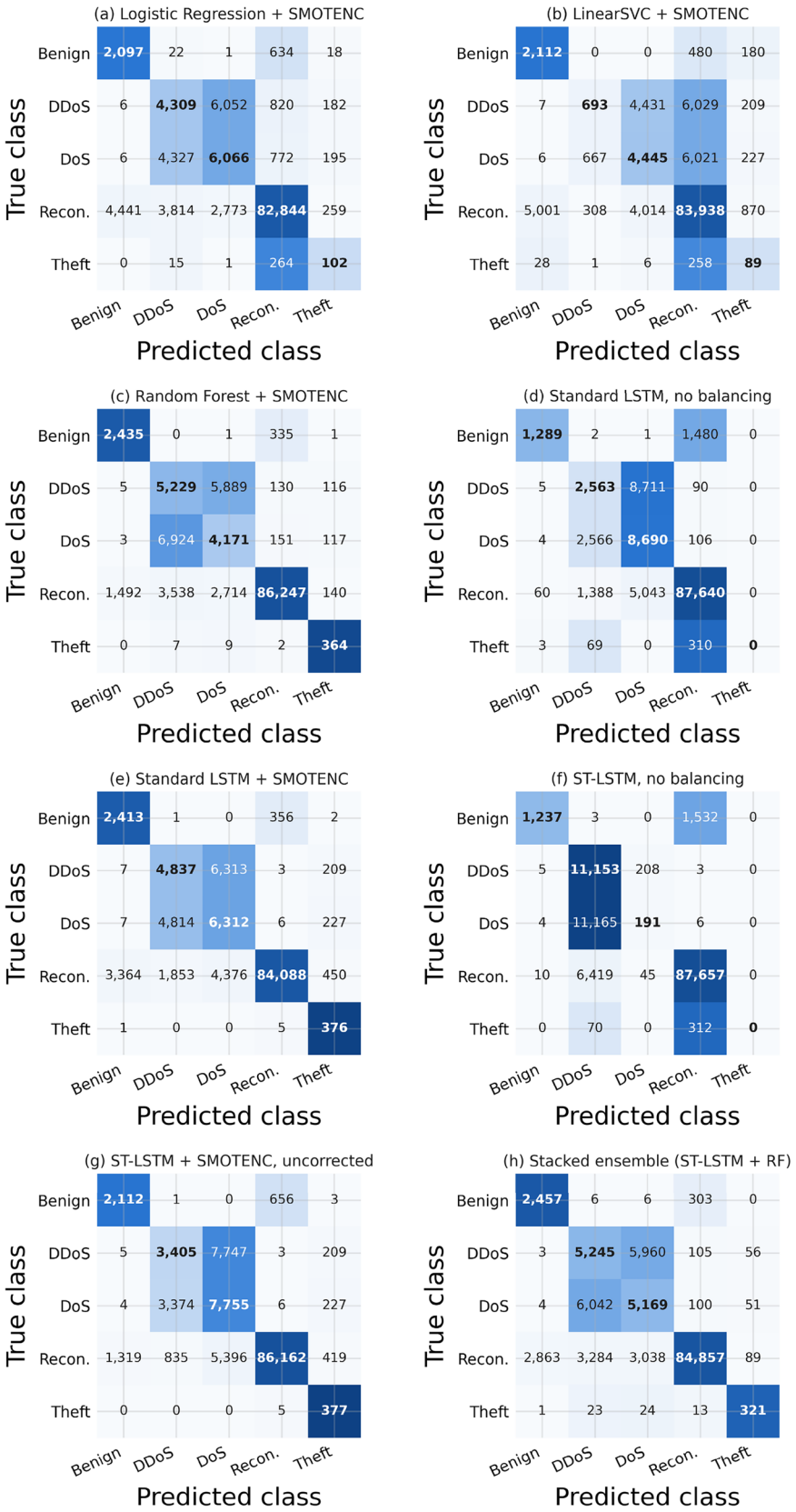


Fig. 10 Test-set confusion matrices of the comparison models: (**a**) Logistic Regression, (**b**) LinearSVC and (**c**) Random Forest trained with SMOTENC, (**d**) single-layer LSTM without and (**e**) with SMOTENC, (**f**) ST-LSTM without balancing, (**g**) ST-LSTM with SMOTENC, (**h**) stacked ensemble.

Table 11 ST-LSTM trained with standard SMOTE and with SMOTENC at the same ratio ($r = 0.25$), before recalibration.

Quantity	Standard SMOTE	SMOTENC
Invalid PROTOCOL values created (valid codes: 3)	42	0
Invalid L7_PROTO values created (valid codes: 87)	1,104	0
Invalid TCP_FLAGS values created (valid codes: 23)	849	0
Accuracy	0.8300	0.8316
Precision (weighted)	0.8296	0.8709
F1 (weighted)	0.8157	0.8435
F1 (macro)	0.5277	0.5876
DDoS F1	0.0000	0.3587
ROC-AUC (weighted/macro)	0.9669/0.9643	0.9670/0.9646

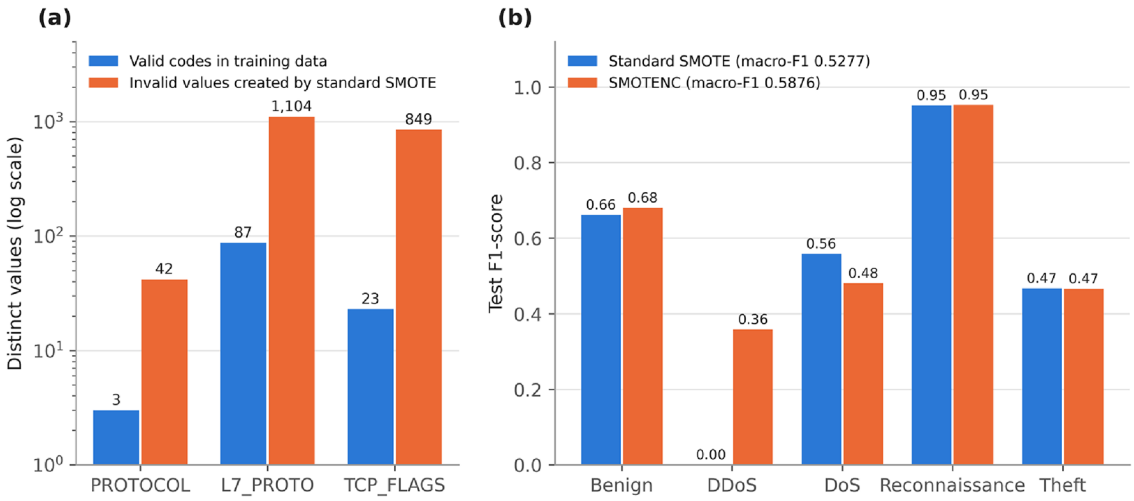


Fig. 11 Standard SMOTE compared with SMOTENC at $r = 0.25$: (a) valid category codes in the training data and invalid values created by standard SMOTE (logarithmic axis), (b) per-class test F1 of the ST-LSTM.

Table 12 Label-conflict analysis of NF-BoT-IoT with the ten predictors.

Quantity	Value
Distinct feature vectors that occur under more than one class	57,206
Flows in such vectors (full corpus)	148,433 (24.73%)
Share of DDoS/DoS flows in conflicting vectors	99.94%/99.96%
Share of Benign/Reconnaissance/Theft flows in conflicting vectors	7.06%/7.12%/15.77%
Attainable five-class accuracy, full corpus/test partition	84.98%/84.91%
Attainable accuracy without L4_SRC_PORT/without both ports (full corpus)	84.10%/83.84%
Attainable accuracy with DDoS and DoS merged, full corpus/test partition	94.45%/94.38%
Test flows in conflicting vectors	29,756 (24.79%); model accuracy 38.37%
Unambiguous test flows	90,264 (75.21%); model accuracy 97.93%
Test DDoS/DoS flows in conflicting vectors	11,365 of 11,369/11,363 of 11,366
Proposed model accuracy relative to the test ceiling	83.98%/84.91% = 98.9%

Label conflicts and the attainable ceiling

The persistent DDoS–DoS confusion raised the question of whether the two classes can be separated at all in this feature space. The label-conflict analysis of Section "Evaluation protocol and label-conflict analysis" answers it (Table 12, Fig. 12). In the full corpus, 57,206 distinct feature vectors occur under more than one class, and

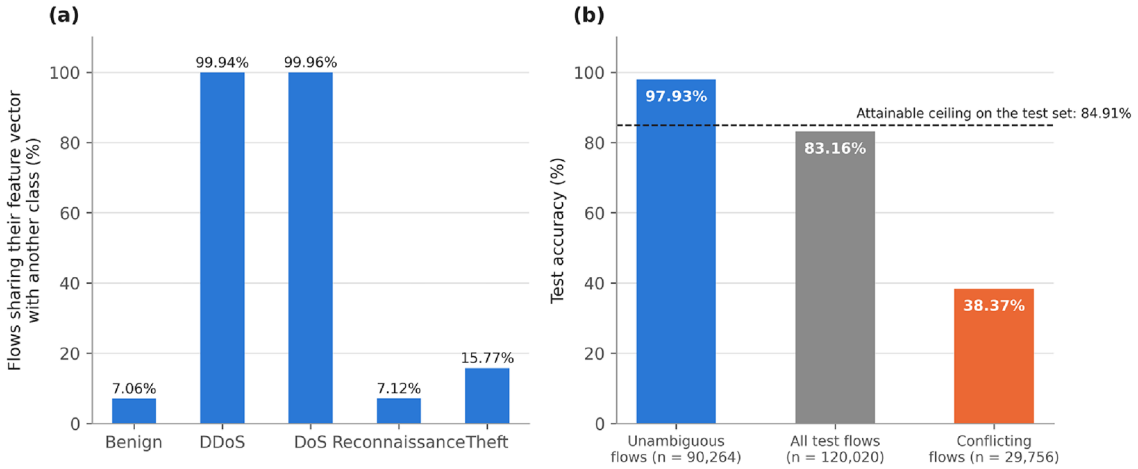


Fig. 12 Label-conflict analysis: (a) share of each class in conflicting feature vectors and (b) test accuracy of the ST-LSTM with SMOTENC.

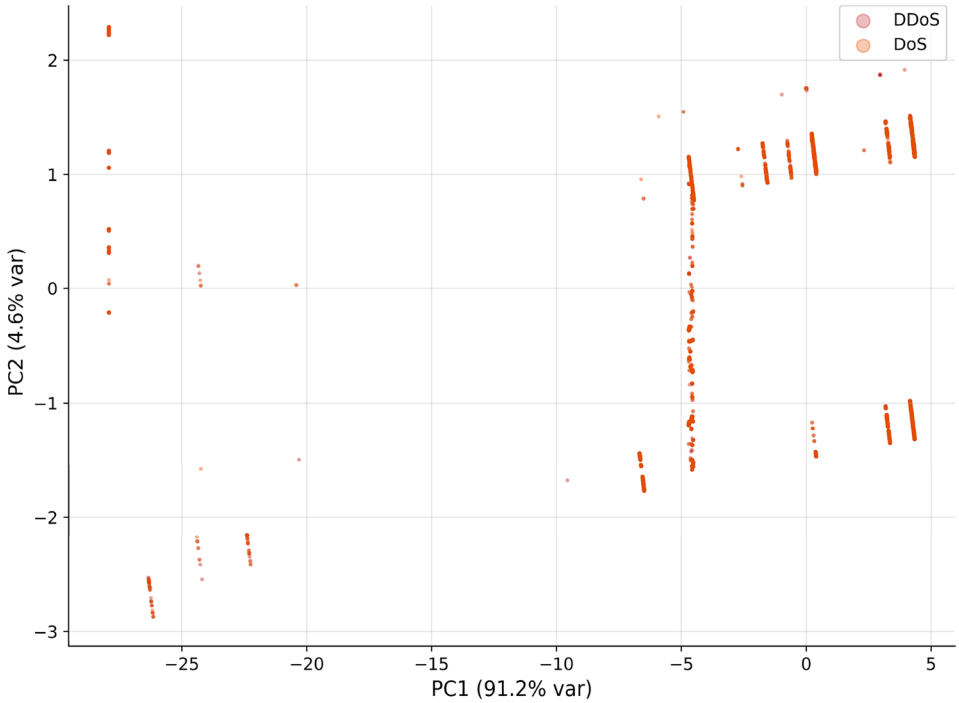


Fig. 13 Principal-component projection of DDoS and DoS flows in the ten-predictor space.

they account for 148,433 flows (24.73%). The conflicts are concentrated in two classes: 99.94% of DDoS flows and 99.96% of DoS flows share their exact feature vector with a flow of another class, compared with 7.06% of Benign, 7.12% of Reconnaissance and 15.77% of Theft flows. DDoS and DoS co-occur in 56,228 vectors, and 31,664 of these also contain Reconnaissance flows. Removing the source port, or both ports, from the comparison leaves the DDoS and DoS shares above 99.9%. The two classes are therefore not separated by the ports or by any other of the ten predictors. The standardised mean differences between DDoS and DoS in the training partition confirm this; Cohen's *d* does not exceed 0.007 for any predictor. Fig. 13 exhibits the two classes on the first two principal components of the ten-predictor space, which explain 91.2% and 4.6% of the variance, the points of the two classes coincide.

Table 13 Supplementary four-class evaluation with DDoS and DoS merged (test partition, n = 120,020).

Configuration	Val. macro F1	Accuracy	F1 (w)	F1 (macro)	ROC-AUC (w/macro)
Before recalibration	0.7420	0.9240	0.9285	0.7408	0.9739/0.9849
Recalibrated with tuned offsets	0.8495	0.9350	0.9366	0.8537	0.9780/0.9862

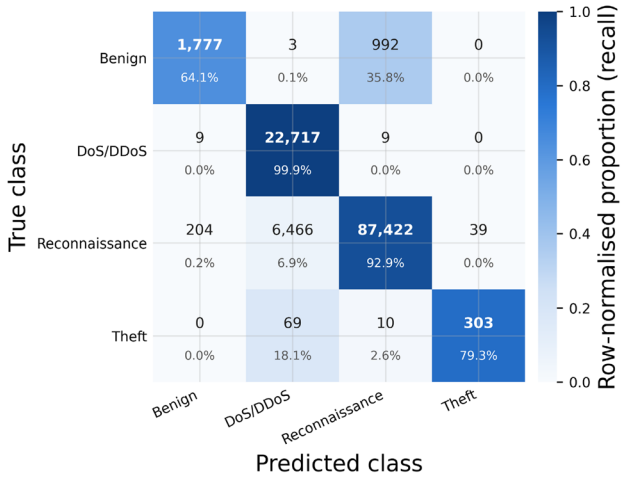


Fig. 14 Test-set confusion matrix of the four-class ST-LSTM with DDoS and DoS merged.

Because identical inputs cannot receive different labels, no classifier restricted to these predictors can exceed 84.98% five-class accuracy on the full corpus, or 84.91% on the test partition. The proposed model reaches 83.98%, which is 98.9% of the attainable value. The error decomposition in Fig. 12(b), computed for the ST-LSTM with SMOTENC before recalibration, shows where the remaining errors lie; the dashed line marks the attainable ceiling of the test partition. The model results in Table 12 refer to the same network unless stated otherwise. On the 90,264 unambiguous test flows, this network is 97.93% accurate, with F1 0.9893 for Reconnaissance, 0.7624 for Theft and 0.6965 for Benign. On the 29,756 conflicting flows, which contain almost every DDoS and DoS flow, it is correct for 38.37%. The overall accuracy of 83.16% is the weighted combination of the two, $0.7521 \times 0.9793 + 0.2479 \times 0.3837$.

The same analysis shows that exact duplicates do not inflate the reported results. Of the test flows, 21.90% have an identical feature vector in the training partition. These flows are harder, not easier, for the model (37.53% accuracy against 95.96% on test flows with no duplicate in training), because most of them belong to the conflicting DDoS, DoS and Reconnaissance vectors.

Supplementary analysis with DDoS and DoS merged

Since DDoS and DoS cannot be distinguished by the ten predictors, the ST-LSTM was also trained on four classes with the two merged, using the same partitions, SMOTENC ratio and settings. The results are given in Table 13, and the confusion matrix is shown in Fig. 14. After recalibration, the four-class model reaches 93.50% accuracy, which is 99.1% of the four-class ceiling of 94.38%, with macro F1 0.8537 and weighted ROC-AUC 0.9780. Per-class F1 is 0.7463 for Benign, 0.8739 for DoS/DDoS, 0.9577 for Reconnaissance and 0.8370 for Theft, and the merged class is recalled almost completely (0.9992). The largest remaining error is the assignment of 6,466 Reconnaissance flows to the merged class, which is consistent with the 6,774 Reconnaissance test flows whose feature vectors also occur under another class. This analysis supplements, and does not replace, the five-class evaluation.

Discussion

The results show that careful protocol design changes both the level and the interpretation of IDS performance on NF-BoT-IoT. With the target and the label-derived and identifier columns excluded, and with the validation and test partitions kept free of synthetic samples, the three-tier ST-LSTM reaches 83.98% accuracy and 0.6609 macro F1. These values are lower than the accuracies of roughly 95% or more that are often reported on related IoT benchmarks[17,22,23,38]. Those studies use different datasets, features and protocols, so their figures are not directly comparable, but the label-conflict analysis explains why the attainable level on NF-BoT-IoT is lower. About a quarter of the flows in NF-BoT-IoT share their exact feature vector with a flow of another class, and nearly all of them belong to DDoS and DoS. No model restricted to these ten predictors can exceed 84.91% accuracy on the test partition. Accuracies above this bound on the same feature set would require information that the predictors do not contain, such as the target or its derivative, host identifiers or synthetic neighbours of test flows, which are the forms of leakage described by *Kapoor and Narayanan*[13] and *Arp et al.*[14].

The distinction between DoS and DDoS lies in the number of sources that send the traffic. This property is visible in the IP addresses, which were removed as host identifiers, but not in the statistics of individual flows. The confusion between the two classes is therefore a property of the representation, not a weakness of a particular model: the ST-LSTM, the single-layer LSTM, the Random Forest and the stacked ensemble all divide the two classes in a similar way (Fig. 10). Separating them would require features computed over several flows, for example the number of distinct sources that contact a destination within a time window. Such aggregates must be computed from training data only and without exposing host identities, so that they do not reintroduce the leakage the protocol is designed to prevent. When the two classes are merged, the model reaches 99.1% of the attainable four-class accuracy.

SMOTENC is the appropriate oversampling method for NetFlow records that combine continuous counts with protocol and flag codes. Standard SMOTE generated almost two thousand category values that do not exist, and the network trained on those data lost the DDoS class entirely while its accuracy stayed at 83%. Distance-based oversamplers that interpolate every feature, including Borderline-SMOTE[42] and ADASYN[43], share this limitation when categorical codes are present. The partial ratio also matters: raising the minority classes to 25% of the majority was better on validation data than 15% or 40%, which shows that more synthetic data are not always better.

Oversampling improves minority recall but also changes the class frequencies that the network learns, so that small classes are over-predicted. Rescaling the outputs to the original class frequencies[16] addressed this at negligible cost and improved macro F1 in every cross-validation fold. Its effect is visible in the precision of the rare classes: Theft precision increased from 0.31 to 0.74 and Benign precision from 0.61 to 0.89. The trade-off is a lower Benign recall, because more borderline benign flows are assigned to Reconnaissance. For an operator, this corresponds to fewer false alarms on rare classes at the cost of some missed benign labels. The offsets can be re-tuned on validation data to favour recall where missed detections are costlier. The stacked ensemble reached a similar cross-validated macro F1 with lower variance, but it adds a second learner and gave lower accuracy and ROC-AUC. The recalibration achieves a comparable effect with the network alone and no additional parameters.

The two-by-two design of depth and balancing refines the role of model capacity. Without balancing, the deeper network was worse than the single-layer LSTM on macro F1 because it collapsed DDoS and DoS into one output and ignored Theft. With SMOTENC it gained almost twice as much macro F1 as the shallow model. Capacity and balancing therefore act together rather than independently, a pattern that aggregate accuracy, which varied by less than two percentage points across the four configurations, does not show.

With 482,053 parameters (about 1.84 MB in 32-bit floating point) and one-time step per flow, the model can score individual NetFlow records as they are exported, and the recalibration adds one vector addition per flow. In privacy-sensitive IoT and medical IoT settings, such a detector would form one component of layered security,

key-management and data-sharing architectures[50,51]. Inference latency and throughput were not measured in this study, so suitability for a specific real-time budget has yet to be established.

Conclusion

This study evaluated a three-tier stacked Time-Distributed LSTM for multiclass intrusion detection on NF-BoT-IoT under a leakage-free protocol. The ten predictors, the target and the excluded columns were specified explicitly; the data were partitioned before any fitting; SMOTENC was applied to the training partition only at a validation-selected ratio; and the network outputs were rescaled to the original class frequencies at inference. On the untouched test set the model reached 83.98% accuracy, 0.8500 weighted F1, 0.6609 macro F1 and 0.9678 weighted ROC-AUC, the best values among the compared models, and five-fold cross-validation gave 83.94 ± 0.19% accuracy and 0.6131 ± 0.0495 macro F1. Standard SMOTE created invalid category values and eliminated DDoS detection, which supports the use of SMOTENC for mixed-type flow records. The label-conflict analysis showed that DDoS and DoS flows share identical feature vectors, that the attainable five-class accuracy is 84.91% on the test partition, and that the model reaches 98.9% of it.

The study has several limitations. First, the DDoS and DoS classes cannot be separated with the ten per-flow predictors, so five-class results on NF-BoT-IoT are bounded by the data, and the merged four-class analysis is supplementary. Second, each flow is scored as a single time step, so the recurrent tiers do not model dependencies across flows. Third, the evaluation uses one dataset and a single test split, complemented by five-fold cross-validation; the cross-validated macro F1 still varies by about 0.05 between folds. Fourth, inference latency, concept drift and adversarial robustness were not examined. Fifth, the ablation of depth and balancing and the comparison of standard SMOTE with SMOTENC were run on a single split.

Future work will extend the model to windows of consecutive flows with multi-flow aggregate features computed without host identifiers, which are needed to separate DoS from DDoS. It will also validate the protocol on further NetFlow benchmarks such as the second version of the NF datasets, UNSW-NB15 and CSE-CIC-IDS2018, and measure inference cost on edge hardware. Explainability methods would help operators interpret the per-class decisions, and federated training would support privacy-sensitive IoT and medical IoT deployments.

Acknowledgements We sincerely appreciate the insightful suggestions, constructive comments, and thoughtful queries from the reviewers and the editor. Their valuable feedback has significantly enriched the manuscript through comprehensive revisions and improvements. We believe that the revised version satisfactorily addresses all the concerns raised and hope that it now meets the expectations of the editorial team of Scientific Reports.

Author contributions A) Mustapha Ismail Kwari led the conceptualization, methodology design, implementation, data curation, formal analysis, and preparation of the original manuscript draft. B) N. Rajkumar provided overall supervision, contributed to methodology refinement, validation, and critical revision of the manuscript. C) C. Selvarathi supported methodology development, validation, and manuscript review. D) Rabiu Aliyu Abdulkadir contributed to data analysis, investigation, visualization, and manuscript review. E) Mideksa Tola Jiru contributed to investigation, data curation, manuscript review, and served as the corresponding author. All authors read and approved the final manuscript.

Data availability The NF-BoT-IoT dataset analysed in this study is publicly available from the University of Queensland NIDS datasets repository (https://staff.itee.uq.edu.au/marius/NIDS_datasets/)[45].

Code availability The code used in this study is available on GitHub (https://github.com/kwari4real-code/ST-LSTM-NF-BoT-IoT) and archived on Zenodo (https://doi.org/10.5281/zenodo.23017817).

Declaration

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A. & Anwar, A. N. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access* **8**, 165130–165150. https://doi.org/10.1109/ACCESS.2020.3022862 (2020).
2. Lone, A. N., Mustajab, S. & Alam, M. A comprehensive study on cybersecurity challenges and opportunities in the IoT world. *Secur. Priv.* **6**(6), e318. https://doi.org/10.1002/spy2.318 (2023).
3. Lansky, J. et al. Deep learning-based intrusion detection systems: A systematic review. *IEEE Access* **9**, 101574–101599. https://doi.org/10.1109/ACCESS.2021.3097247 (2021).
4. Kasongo, S. M. A deep learning technique for intrusion detection system using a recurrent neural networks based framework. *Comput. Commun.* **199**, 113–125. https://doi.org/10.1016/j.comcom.2022.12.010 (2023).
5. Bagui, S. & Li, K. Resampling imbalanced data for network intrusion detection datasets. *Journal of Big Data* **8**(1), 6 (2021).
6. Misrak, S. F. & Melaku, H. M. Lightweight intrusion detection system for IoT with improved feature engineering and advanced dynamic quantization. *Discov. Internet Things* https://doi.org/10.1007/s43926-025-00203-8 (2025).
7. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9**(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735 (1997).
8. Laghrissi, F., Douzi, S., Douzi, K. & Hssina, B. Intrusion detection systems using long short-term memory (LSTM). *J. Big Data* **8**(1), 65 (2021).
9. Fu, Y., Du, Y., Cao, Z., Li, Q. & Xiang, W. A deep learning model for network intrusion detection with imbalanced data. *Electronics* **11**(6), 898 (2022).
10. Wu, T. et al. Intrusion detection system combined enhanced random forest with SMOTE algorithm. *EURASIP Journal on Advances in Signal Processing* **1**, 2022. https://doi.org/10.1186/s13634-022-00871-6 (2022).
11. M. I. Kwari and N. Rajkumar, "An enhanced voting ensemble model for IoT intrusion detection using the CICIoT2023 dataset," in *Proc. Ganitara Int. Computing and AI Conf. 2025*, pp. 67–77, https://doi.org/10.63169/IGCAI2025.p9 (2025).
12. M. I. Kwari, N. Rajkumar, and C. Selvarathi, "Optimized multiclass classification of IoT cyberattacks using a hybrid Random Forest–Gradient Boosting stacking ensemble," in press, (2026).
13. Kapoor, S. & Narayanan, A. "Leakage and the reproducibility crisis in machine-learning-based science,". *Patterns* **4**(9), 100804. https://doi.org/10.1016/j.patter.2023.100804 (2023).
14. D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, pp. 3971–3988, (2022).
15. Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. SMOTE: Synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **16**, 321–357. https://doi.org/10.1613/jair.953 (2002).
16. Saerens, M., Latinne, P. & Decaestecker, C. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Comput.* **14**(1), 21–41. https://doi.org/10.1162/089976602753284446 (2002).
17. Halbouni, A. et al. CNN-LSTM: Hybrid deep neural network for network intrusion detection system. *IEEE Access* **10**, 99837–99849 (2022).
18. Altunay, H. C. & Albayrak, Z. "A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks,". *Eng. Sci. Technol. Int. J.* **38**, 101322. https://doi.org/10.1016/j.jestch.2022.101322 (2023).
19. Imrana, Y., Xiang, Y., Ali, L. & Abdul-Rauf, Z. A bidirectional LSTM deep learning approach for intrusion detection. *Expert Syst. Appl.* **185**, 115524. https://doi.org/10.1016/j.eswa.2021.115524 (2021).
20. Ben Said, R., Sabir, Z. & Askerzade, I. CNN-BiLSTM: A hybrid deep learning approach for network intrusion detection system in software-defined networking with hybrid feature selection. *IEEE Access* **11**, 138732–138747 (2023).
21. Chen, S. & Feng, X. A novel intrusion detection framework using hybrid deep learning to detect IIoT cloud environments attacks. *Sci. Rep.* **16**, 11666. https://doi.org/10.1038/s41598-025-15419-5 (2026).

22. Alashjaee, A. M. Deep learning for network security: An Attention-CNN-LSTM model for accurate intrusion detection. *Sci. Rep.* **15**, 21856 (2025).
23. Dash, N. et al. An optimized LSTM-based deep learning model for anomaly network intrusion detection. *Sci. Rep.* **15**, 1554. https://doi.org/10.1038/s41598-025-85248-z (2025).
24. Karthic, S. & Kumar, S. M. Hybrid optimized deep neural network with enhanced conditional random field based intrusion detection on wireless sensor network. *Neural Process. Lett.* **55**, 459–479. https://doi.org/10.1007/s11063-022-10892-9 (2023).
25. Sundaram, K., Subramanian, S., Natarajan, Y., Perumalsamy, A. & Yuvaraj, N. Improving performance of intrusion detection using ALO selected features and GRU network. *SN Computer Science* **4**, 809. https://doi.org/10.1007/s42979-023-02311-0 (2023).
26. S. Suhana, S. Karthic, and N. Yuvaraj, "Ensemble based dimensionality reduction for intrusion detection using random forest in wireless networks," in *Proc. 5th Int. Conf. Smart Systems and Inventive Technology (ICSSIT)*, IEEE, pp. 704–708, 2023, https://doi.org/10.1109/ICSSIT55814.2023.10060929.
27. Sundaram, K. & Kumar, S. M. Advanced GWORNN based intrusion detection scheme for securing wireless sensor networks. *Wirel. Pers. Commun.* https://doi.org/10.1007/s11277-026-12145-7 (2026).
28. Sundaram, K., Karthic, S., Natarajan, Y., Perumalsamy, A. & Yusuf Ali, A. A. A novel hybrid feature selection with cascaded LSTM: Enhancing security in IoT networks. *Wireless Commun. Mobile Comput.* **2024**, 5522431. https://doi.org/10.1155/2024/5522431 (2024).
29. Sumathi, S. A scalable and explainable hybrid learning framework for cyber threat detection in SDN environments. *Iran. J. Sci. Technol. Trans. Electr. Eng.* https://doi.org/10.1007/s40998-026-01129-9 (2026).
30. Sumathi, S., Rajesh, R. & Lim, S. Intelligent intrusion detection system for IoT networks: A feature-enhanced deep belief network approach. *IEEE Internet Things J.* **13**(3), 4477–4491. https://doi.org/10.1109/JIOT.2025.3633297 (2026).
31. Alasmari, S. M., Sakly, H., Kraiem, N. & Algarni, A. Phishing detection in IoT: An integrated CNN-LSTM framework with explainable AI and LLM-enhanced analysis. *Discov. Internet Things* https://doi.org/10.1007/s43926-025-00202-9 (2025).
32. Sharma, N. & Shambharkar, P. G. Multi-attention DeepCRNN: An efficient and explainable intrusion detection framework for Internet of Medical Things environments. *Knowl. Inf. Syst.* **67**, 5783–5849. https://doi.org/10.1007/s10115-025-02402-9 (2025).
33. Elnakib, O., Shaaban, E., Mahmoud, M. & Emara, K. EIDM: Deep learning model for IoT intrusion detection systems. *J. Supercomput.* **79**(12), 13241–13261 (2023).
34. Kantharaju, V. et al. Machine learning based intrusion detection framework for detecting security attacks in internet of things. *Sci. Rep.* **14**, 30275. https://doi.org/10.1038/s41598-024-81535-3 (2024).
35. Qaddos, A. et al. A novel intrusion detection framework for optimizing IoT security. *Sci. Rep.* **14**, 21789. https://doi.org/10.1038/s41598-024-72049-z (2024).
36. Zhang, H. Development of an intelligent intrusion detection system for IoT networks using deep learning. *Discover Internet of Things* **5**, 74. https://doi.org/10.1007/s43926-025-00177-7 (2025).
37. Di Mauro, M., Galatro, G., Fortino, G. & Liotta, A. Supervised feature selection techniques in network intrusion detection: A critical review. *Eng. Appl. Artificial Intel.* **101**, 104216 (2021).
38. Sayegh, H. R., Dong, W. & Al-madani, A. M. Enhanced intrusion detection with LSTM-based model, feature selection, and SMOTE for imbalanced data. *Appl. Sci.* **14**(2), 479. https://doi.org/10.3390/app14020479 (2024).
39. Joloudari, J. H., Marefat, A., Nematollahi, M. A., Oyelere, S. S. & Hussain, S. Effective class-imbalance learning based on SMOTE and convolutional neural networks. *Appl. Sci.* **13**(6), 4006. https://doi.org/10.3390/app13064006 (2023).
40. Qaddoura, R., Al-Zoubi, A. M., Almomani, I. & Faris, H. A multi-stage classification approach for IoT intrusion detection based on clustering with oversampling. *Appl. Sci.* **11**(7), 3022. https://doi.org/10.3390/app11073022 (2021).
41. Eid, A. M., Soudan, B., Nassif, A. B. & Injadat, M. N. Comparative study of ML models for IIoT intrusion detection: Impact of data preprocessing and balancing. *Neural Comput. Appl.* **36**(19), 11661. https://doi.org/10.1007/s00521-024-09841-5 (2024).
42. H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intelligent Computing (ICIC)*, Springer, pp. 878–887, 2005.
43. H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN)*, pp. 1322–1328, 2008.
44. Koroniotis, N., Moustafa, N., Sitnikova, E. & Turnbull, B. Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset. *Future Gener. Comput. Syst.* **100**, 779–796 (2019).
45. M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Proc. Int. Conf. Big Data Technologies and Applications (BDTA)*, Springer, pp. 117–135, 2020.
46. Pedregosa, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011).
47. Lemaître, G., Nogueira, F. & Aridas, C. K. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *J. Mach. Learn. Res.* **18**(17), 1–5 (2017).

48. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.
49. T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, pp. 2980–2988, 2017.
50. Sharma, N. & Shambharkar, P. G. Multi-layered security architecture for IoMT systems: Integrating dynamic key management, decentralized storage, and dependable intrusion detection framework. *Int. J. Mach. Learn. Cybern.* **16**, 6399–6446. https://doi.org/10.1007/s13042-025-02628-7 (2025).
51. Sharma, N. & Shambharkar, P. G. Blockchain-based framework for secure medical data sharing and disease diagnosis using optimized deep belief networks. *Clust. Comput.* **28**, 1029. https://doi.org/10.1007/s10586-025-05706-1 (2025).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Mustapha Ismail Kwari[1] · N. Rajkumar[1] · C. Selvarathi[1] · Rabiu Aliyu Abdulkadir[2] · Mideksa Tola Jiru[3]

✉ Mustapha Ismail Kwari
vtd1224@veltech.edu.in

✉ Mideksa Tola Jiru
mideksatol@gmail.com

[1] Department of Computer Science and Engineering, School of Computing, Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, Tamil Nadu 600 062, Chennai, India

[2] Department of Computer Science, School of Computer Science and Technology, Shandong Xiehe University, Jinan, Shandong, China

[3] Department of Mathematics, Arsi University, Asella, Ethiopia